MLFP mlfp04 — Speaker Notes

- Module 4: Unsupervised Machine Learning and Advanced Techniques for Insights — Speaker Notes
  - Slide 1: Module 4 Title — Unsupervised ML and Advanced Techniques for Insights
  - Slide 2: Recap — Your Journey So Far
  - Slide 3: The Feature Engineering Spectrum
  - Slide 4: Module 4 Road Map
  - Slide 5: 4.1 Clustering — Lesson Title
  - Slide 6: K-Means — The Workhorse
  - Slide 7: Choosing k — The Elbow Method
  - Slide 8: Hierarchical Clustering
  - Slide 9: DBSCAN and HDBSCAN
  - Slide 10: Spectral Clustering
  - Slide 11: Cluster Evaluation — Internal Metrics
  - Slide 12: Cluster Evaluation — External Metrics
  - Slide 13: Algorithm Selection Guide
  - Slide 14: Application — Customer Segmentation
  - Slide 15: Kailash Bridge — AutoMLEngine for Clustering
  - Slide 16: Exercise 4.1 — Customer Segmentation
  - Slide 17: 4.2 EM Algorithm and Gaussian Mixture Models — Lesson Title
  - Slide 18: Hard vs Soft Clustering
  - Slide 19: The EM Algorithm — E-Step
  - Slide 20: The EM Algorithm — M-Step
  - Slide 21: EM Convergence and Log-Likelihood
  - Slide 22: GMM — EM Applied to Gaussians
  - Slide 23: Mixture of Experts — Modern Application
  - Slide 24: Exercise 4.2 — EM from Scratch
  - Slide 25: 4.3 Dimensionality Reduction — Lesson Title
  - Slide 26: PCA Step 1 — Decorrelate
  - Slide 27: PCA Step 2 — Reduce
  - Slide 28: PCA via SVD
  - Slide 29: Reconstruction Error
  - Slide 30: Kernel PCA
  - Slide 31: t-SNE — Visualisation Specialist
  - Slide 32: UMAP — The Modern Default
  - Slide 33: Manifold Learning — Reference Table
  - Slide 34: Intrinsic Dimensionality
  - Slide 35: Exercise 4.3 — PCA, t-SNE and UMAP
  - Slide 36: 4.4 Anomaly Detection — Lesson Title
  - Slide 37: Statistical Outlier Detection
  - Slide 38: Isolation Forest
  - Slide 39: Local Outlier Factor (LOF)
  - Slide 40: Score Blending — Combining Detectors
  - Slide 41: Kailash Bridge — EnsembleEngine for Anomaly Detection
  - Slide 42: Anomaly Detection in Production
  - Slide 43: Exercise 4.4 — Financial Anomaly Detection
  - Slide 44: 4.5 Market Basket Analysis — Lesson Title
  - Slide 45: Association Rules — The Metrics
  - Slide 46: Apriori vs FP-Growth
  - Slide 47: From Rules to Features
  - Slide 48: Applications Beyond Retail
  - Slide 49: Exercise 4.5 — Market Basket Analysis

- Slide 50: 4.6 NLP — Text to Topics — Lesson Title
- Slide 51: Text as Data — Representation
- Slide 52: TF-IDF — Derivation
- Slide 53: Word Embeddings — Tools, Not Derivation
- Slide 54: LDA — Latent Dirichlet Allocation
- Slide 55: NMF and BERTopic
- Slide 56: Topic Coherence — Evaluating Quality
- Slide 57: Sentiment Analysis — Text Classification
- Slide 58: Exercise 4.6 — Topic Modelling and Sentiment
- Slide 59: 4.7 Recommender Systems — Lesson Title
- Slide 60: Content-Based vs Collaborative Filtering
- Slide 61: User-Based vs Item-Based CF
- Slide 62: Matrix Factorisation — The Core Idea
- Slide 63: ALS — Alternating Least Squares
- Slide 64: Implicit Feedback and Hybrid Systems
- Slide 65: THE PIVOT — Optimisation Drives Feature Discovery
- Slide 66: Visualising Learned Embeddings
- Slide 67: Exercise 4.7 — Recommender Systems
- Slide 68: 4.8 DL Foundations — Lesson Title
- Slide 69: The Bridge — From Matrix Factorisation to Neural Networks
- Slide 70: Neural Network Architecture
- Slide 71: Forward Pass
- Slide 72: Linear Regression as a Neural Network
- Slide 73: Loss Functions and Gradient Descent
- Slide 74: Backpropagation — Chain Rule Through Layers
- Slide 75: Hidden Layers = Automated Feature Engineering
- Slide 76: Activation Functions
- Slide 77: Dropout and Batch Normalisation
- Slide 78: Weight Initialisation
- Slide 79: Optimisers — From SGD to Adam
- Slide 80: Loss Functions Taxonomy
- Slide 81: Learning Rate Schedules and Early Stopping
- Slide 82: Regression vs Classification Output
- Slide 83: Kailash Bridge — OnnxBridge for Neural Networks
- Slide 84: Exercise 4.8 — Neural Network from Scratch
- Slide 85: Module 4 — The Complete Arc
- Slide 86: Module 4 Assessment
- Slide 87: Looking Ahead — Module 5
- Slide 88: Module 4 Complete

Module 4: Unsupervised Machine Learning and Advanced Techniques for Insights — Speaker Notes

Total time: ~180 minutes (3 hours) Audience: working professionals; instructors must scaffold for both novices who just finished M3 and ML practitioners who have used sklearn for years.

Module 4 is THE pivot of the MLFP curriculum. The Feature Engineering Spectrum (from the design principles) is the spine of the entire three hours: manual feature engineering (M1–M3) on the left, unsupervised discovery (4.1–4.6) in the middle, the optimisation-driven pivot (4.7), and deep learning (4.8) on the right. Every lesson returns to this spectrum. Instructors should draw it on the whiteboard at the start and point back to it before every new lesson.

Slide 1: Module 4 Title — Unsupervised ML and Advanced Techniques for Insights

Time: ~2 min **Talking points**: - Welcome the class back. Read the provocation aloud: “The algorithm that found $2 billion in hidden fraud had never seen a single labelled example.” - Let it sit. Ask the room: “How is it possible to find something you were never told to look for?” - Frame the module: “For three modules you have worked with labelled data. Today those labels disappear, and by the end of the day, the features will engineer themselves.” - “If beginners look confused”: “Do not worry about the maths yet. We will walk through one algorithm at a time, and each one has a Kailash engine you can call with two lines of code.” - “If experts look bored”: “This module is not a grab-bag of clustering algorithms. It is a single continuous story that ends with a neural network you built from scratch — and a clear theoretical bridge from K-means to backpropagation.” **Transition**: “Let me remind you where we have been so you can see where we are going.”

Slide 2: Recap — Your Journey So Far

Time: ~2 min **Talking points**: - Walk down the table fast: M1 gave you data pipelines, M2 gave you statistical foundations, M3 gave you supervised ML with labelled data. - Stop on the callout: “Everything so far needed either a human to craft features or a target label. Today those constraints vanish.” - Emphasise that M3 already used DataExplorer, PreprocessingPipeline, TrainingPipeline, and ModelRegistry. Today adds AutoMLEngine, EnsembleEngine, ModelVisualizer, and OnnxBridge — every new technique in M4 has a matching Kailash engine. - “If beginners look confused”: “You have already done the hardest part — you built working supervised models in M3. Today is about relaxing assumptions, not learning something new from scratch.” - “If experts look bored”: “The M3 → M4 jump mirrors the Hastie / Tibshirani / Friedman ordering: supervised before unsupervised. The pedagogical reason is that the evaluation instinct from M3 transfers directly to cluster quality and topic coherence today.” **Transition**: “Here is the single slide that organises everything you will learn today.”

Slide 3: The Feature Engineering Spectrum

Time: ~4 min **Talking points**: - Slow down. This is the most important slide in the module. - Point to the three nodes in order: Manual (M1–M3), USML (today), Deep Learning (4.8 and M5). Read the tagline under each: “Human crafts features”, “Algorithm discovers features”, “Architecture learns features”. - Make the counts explicit: manual is n → n (you pick the features), clustering and topic models are n → 1 or n → k (the algorithm compresses), deep learning is n → m (the network invents a new feature space whose size you choose). - Key line to repeat verbatim: “USML is not a separate discipline. It is the bridge between hand-crafted features and deep-learned representations.” - Promise the room: “By lesson 4.7 I will show you the exact moment when optimisation starts discovering features on its own. By lesson 4.8 you will build a neural network from scratch and see that hidden layer activations ARE embeddings, just like the ones collaborative filtering learned.” - “If beginners look confused”: “Think of the spectrum as three ways to answer the question: who decides what a good feature is? You, the algorithm, or the architecture? Today we move left to right.” - “If experts look bored”: “This is Bengio’s representation learning argument, compressed. Every lesson today is a point on that spectrum, and the pivot at 4.7 is where matrix factorisation meets gradient descent.” **Transition**: “Here is the eight-lesson road map that walks us across the spectrum.”

Slide 4: Module 4 Road Map

Time: ~2 min **Talking points**: - Walk the eight lessons quickly. Do not linger on any one — this is the preview. - Call out the arc: “4.1 through 4.6 all discover features WITHOUT optimisation. 4.7 is the first time optimisation discovers features. 4.8 generalises optimisation-driven feature discovery to any non-linear function.” - Point at

4.7 and 4.8 and say: “These are the two slides the rest of the module builds towards. Everything before them is setup; everything after them is scale-up.” - “If beginners look confused”: “Eight topics sound like a lot. They are not eight disconnected ideas — they are eight chapters in one story, and the story is: how do features come into existence when nobody creates them by hand?” - “If experts look bored”: “Notice the ordering deviates from the usual textbook (clustering → DR → anomaly → rules → NLP → recommenders → DL). That ordering is deliberate — it moves from concrete group discovery to increasingly sophisticated latent-factor models, ending at the pivot.” **Transition**: “Lesson 4.1. Clustering. The simplest form of unsupervised learning.”

Slide 5: 4.1 Clustering — Lesson Title

Time: ~1 min **Talking points**: - Read the subtitle: “Discovering group structure without labels.” - Contrast with M3: “In M3 you predicted outcomes using labelled data. Now: what if there are no labels? Clustering is the answer that asks the data to organise itself.” - Set the lesson tempo: 12 slides, ~25 minutes. You will meet five algorithms and two families of evaluation metrics. **Transition**: “Start with the workhorse — K-means.”

Slide 6: K-Means — The Workhorse

Time: ~3 min **Talking points**: - Walk the four steps on screen slowly: choose k centroids, assign points to nearest centroid, recompute centroids as cluster means, repeat until stable. - If the room allows, draw the two-step “assign then update” loop on the whiteboard with three synthetic points and two centroids. - Read the objective aloud: “Minimise the sum over clusters of the sum over points of the squared distance from each point to its centroid.” Emphasise squared — this is why K-means is sensitive to outliers. - Stop on the warning: “Convergence is guaranteed but only to a LOCAL minimum. Different random starts give different answers. k-means++ spreads the initial centroids apart so bad starts are rare.” - Singapore angle: “Imagine clustering HDB resale transactions by price and floor area. K-means will find roughly four groups: small old flats, small new flats, big old flats, big new flats. You did not define those groups — the algorithm did.” - “If beginners look confused”: “Think of a playground with three teachers. Each child goes to the nearest teacher. Then the teachers walk to the middle of their group. Then the children look again. Repeat until nobody moves. That is K-means.” - “If experts look bored”: “The Lloyd iteration is coordinate descent on the squared-error objective, which is why it only finds local optima. The k-means++ seeding achieves an O(log k) expected approximation ratio — see Arthur and Vassilvitskii 2007.” **Transition**: “K-means asks you for k. How do you choose it?”

Slide 7: Choosing k — The Elbow Method

Time: ~2 min **Talking points**: - Describe the elbow method: plot inertia (the K-means objective J) against k. Look for the kink where adding more clusters stops helping. - Admit the limitation: “The elbow is subjective. Two analysts can look at the same plot and pick different k. That is why we always combine it with silhouette and gap statistics.” - Point to the theory callout: “Gap statistic compares your within-cluster dispersion to a null reference distribution. It gives you a principled answer rather than a visual guess.” - “If beginners look confused”: “It is like buying groceries. At some point, one more item makes almost no difference to how full your bag looks. That point is the elbow.” - “If experts look bored”: “In practice, model selection on k should use gap statistic or BIC on a Gaussian mixture (coming in 4.2) — both are more principled than silhouette.” **Transition**: “K-means forces every point into a round cluster. What if your clusters are not round?”

Slide 8: Hierarchical Clustering

Time: ~3 min **Talking points**: - Walk the agglomerative algorithm: start with each point as its own cluster, repeatedly merge the two closest clusters, stop when one cluster remains. - Hit each linkage method with a physical analogy. Single linkage: "nearest neighbours touch" — produces chains. Complete linkage: "worst case dominates" — produces compact spheres. Average: balanced. Ward's: "minimise total variance increase" — usually the best default. - Emphasise dendrograms: "You do not have to choose k in advance. You run the algorithm once, look at the tree, and cut it at the height that makes business sense." - Singapore angle: "Imagine merging Singapore planning areas by demographic similarity. The dendrogram tells you: which two areas are most similar, which larger groupings emerge, and where the natural cuts are." - "If beginners look confused": "A dendrogram is a family tree for your data. You can look at it and decide how far back to group — grandparents, great-grandparents, or all the way back to one ancestor." - "If experts look bored": "Ward's method is equivalent to minimising the error sum of squares at each merge — it is K-means-compatible and is typically the best starting point on Euclidean data." **Transition**: "K-means and hierarchical both assume clusters are blobs. DBSCAN drops that assumption."

Slide 9: DBSCAN and HDBSCAN

Time: ~3 min **Talking points**: - DBSCAN key idea: clusters are dense regions of points, anything sparse is noise. Walk the three categories: core point (has enough neighbours within epsilon), border point (near a core point), noise point (neither). - Strengths: finds clusters of any shape, identifies noise explicitly, does not require k. - Pain: choosing epsilon is hard, and DBSCAN fails on varying-density data. - HDBSCAN: hierarchical extension that auto-selects epsilon, handles varying densities, returns cluster membership probabilities. "In practice, HDBSCAN has replaced DBSCAN almost entirely — it is one of the most reliable clustering algorithms you can use." - Singapore angle: "If you cluster Grab ride pickup points across Singapore, the density varies wildly — Orchard is dense, Tuas is sparse. HDBSCAN handles both in one pass; DBSCAN would force you to choose one epsilon and miss one region." - "If beginners look confused": "Imagine dropping marbles on a map. Wherever they pile up is a cluster. Wherever they are scattered is noise. DBSCAN and HDBSCAN just measure how densely packed your marbles are." - "If experts look bored": "HDBSCAN constructs a minimum spanning tree of mutual reachability distances, builds a cluster hierarchy, and extracts clusters by maximising stability — see Campello, Moulavi, Sander 2013." **Transition**: "DBSCAN still fails on concentric rings. For that we need spectral clustering."

Slide 10: Spectral Clustering

Time: ~2 min **Talking points**: - Theory layer. Do not derive; explain the intuition. Spectral clustering transforms your data into a space where K-means works, even on non-convex shapes. - Three steps: build a similarity graph, compute the graph Laplacian L = D - W, take the bottom-k eigenvectors as new features, run K-means in that new space. - Payoff: handles concentric circles, interleaving spirals, and other geometries that break every other clustering algorithm on this list. - Cost: O(n^3) for the eigendecomposition. "Spectral clustering is the right answer when shape matters and n is small to medium. For n > 50,000 it becomes painful." - "If beginners look confused": "Think of it as changing the view angle. From one angle, the clusters look impossible to separate. Spectral clustering finds the angle where they look separable, then uses K-means there." - "If experts look bored": "The connection to graph partitioning is through normalised cut (Shi and Malik 2000). The eigenvectors of the normalised Laplacian are a relaxation of the binary cut indicator." **Transition**: "You have five algorithms now. How do you know if a clustering is any good?"

Slide 11: Cluster Evaluation — Internal Metrics

Time: ~3 min **Talking points**: - Internal metrics do not need labels — they measure cluster quality from the geometry alone. - Silhouette score: for each point, compare its average distance to its own cluster (a) vs its

average distance to the nearest OTHER cluster (b). s = (b - a) / max(a, b). Range [-1, 1]. Values near 1 mean a point is well inside its cluster; values near 0 mean it sits on a border; negative values mean it probably belongs to a different cluster. - Davies-Bouldin index: average similarity between each cluster and its most similar neighbour. Lower is better. - Calinski-Harabasz: ratio of between-cluster dispersion to within-cluster dispersion. Higher is better. - "If beginners look confused": "Silhouette is the most intuitive. A silhouette near 1 says the point is surrounded by its own cluster. Near 0 says it is on the fence. Negative says it is in the wrong cluster." - "If experts look bored": "All three internal metrics prefer compact, well-separated convex clusters, which is why they systematically under-rate DBSCAN and HDBSCAN results on non-convex data. Always pair them with visual inspection." **Transition**: "When you do have labels — say, you are validating a clustering against a known ground truth — external metrics are available."

Slide 12: Cluster Evaluation — External Metrics

Time: ~2 min **Talking points**: - External metrics compare a clustering against a known reference labelling. Use them during validation, never during clustering itself (otherwise you are doing supervised learning). - ARI (Adjusted Rand Index): agreement between two labellings, corrected for chance. Range [-1, 1]. 1 = identical. 0 = random. - NMI (Normalised Mutual Information): information-theoretic measure of shared structure. Range [0, 1]. - Use case: "You cluster customers with K-means. Marketing already has a segmentation. ARI tells you how much overlap there is." - "If beginners look confused": "Think of ARI as: of all the pairs of customers in the data, what fraction does my clustering agree about with the reference?" - "If experts look bored": "NMI is normalised by entropy, so it handles clusters of very different sizes better than ARI. But NMI is biased towards fine-grained clusterings — adjust with AMI (Adjusted Mutual Information) if you compare at different k." **Transition**: "With five algorithms and two families of metrics, how do you choose?"

Slide 13: Algorithm Selection Guide

Time: ~2 min **Talking points**: - Walk the decision rule of thumb: round convex clusters with known k → K-means. Unknown k and interpretability matters → hierarchical with a dendrogram. Arbitrary shapes, noise points, variable density → HDBSCAN. Non-convex geometry → spectral. - Remind the room this is a starting rule, not a law. Always try two algorithms and compare silhouette. - "If beginners look confused": "When in doubt, try K-means first because it is fast, then try HDBSCAN because it is flexible. If both agree, you are done. If they disagree, look at the data and decide which one matches reality." - "If experts look bored": "Empirically, for tabular data, HDBSCAN with default parameters and silhouette-validated K-means cover 90% of real use cases. Spectral and kernel clustering are reserved for image-like or graph-like data." **Transition**: "Let us apply this to the canonical unsupervised use case."

Slide 14: Application — Customer Segmentation

Time: ~3 min **Talking points**: - Customer segmentation is the default real-world clustering application. Telcos, retailers, banks — everyone does it. - Walk through the typical pipeline: engineer RFM features (Recency, Frequency, Monetary), scale them, run K-means, label each cluster with a human-readable persona ("high-value loyalist", "at-risk lapser", "new explorer"). - Business interpretation is 80% of the value. A cluster labelled "cluster 3" is worthless; a cluster labelled "high-frequency low-value young customers in the west zone" can drive a campaign. - Singapore angle: "A local grocery chain clustering loyalty card members by basket composition and visit frequency finds clusters that map to HDB vs condo residents, young families vs retirees. Those segments drive targeted promotions." - "If beginners look confused": "Segmentation is what happens when marketing asks: who are our customers really? Clustering gives them groups they can see, name, and target." - "If experts look bored": "The honest hard part is feature engineering. Cluster quality depends more on

what you feed the algorithm than which algorithm you run." **Transition**: "Kailash wraps this entire pattern behind one engine."

Slide 15: Kailash Bridge — AutoMLEngine for Clustering

Time: ~3 min **Talking points**: - AutoMLEngine handles the boring parts: algorithm sweep, hyperparameter search, internal metric computation, best-model selection. - Show the code block on screen. Point out that the same engine that did supervised search in M3 now does unsupervised search — you pass `task="clustering"` and a metric. - Philosophy: "You learned K-means and HDBSCAN by hand so you know what the engine is doing under the hood. In production, the engine saves you hours. You do not give up theory — you automate the tedious search." - Remind the class AutoMLEngine is polars-native. No pandas anywhere. - "If beginners look confused": "Think of AutoMLEngine as a taste-test robot. You pour in your data and it tries every recipe in its cookbook, then hands back the best tasting one." - "If experts look bored": "Under the hood it uses HyperparameterSearch with either random search or Bayesian optimisation, and it tracks every trial in ExperimentTracker so you can audit the sweep." **Transition**: "Time to get your hands dirty."

Slide 16: Exercise 4.1 — Customer Segmentation

Time: ~2 min **Talking points**: - Describe the task: Singapore retail transaction data. Compute RFM features. Run K-means, hierarchical, and HDBSCAN. Evaluate with silhouette and DB index. Name each cluster with a business persona. - The assessment criterion is not speed — it is interpretation. "I would rather see three well-labelled clusters than twenty unnamed ones." - Remind students which format to use (local .py, Jupyter, or Colab) and where to find the dataset via `shared.data_loader`. - "If beginners look confused": "The exercise walks you through step by step. You do not have to write the loop yourself — the scaffolding gives you the structure." **Transition**: "Before we break, one more clustering idea. What if a point belongs to multiple clusters at once?"

Slide 17: 4.2 EM Algorithm and Gaussian Mixture Models — Lesson Title

Time: ~1 min **Talking points**: - Read the subtitle: "Soft clustering — probabilistic assignment to groups." - Motivate the lesson: "K-means gives every point exactly one cluster. That is wrong for customers who shop both as a family and as an individual. GMMs give you probabilities." - Preview the 20-line EM implementation you will build from scratch. **Transition**: "The core distinction: hard vs soft."

Slide 18: Hard vs Soft Clustering

Time: ~3 min **Talking points**: - Hard clustering (K-means, DBSCAN): every point gets exactly one label. - Soft clustering (GMM): every point gets a probability distribution over clusters. A customer might be 70% "family shopper" and 30% "convenience shopper". - Why this matters: soft probabilities let you weight features, compute expected values across segments, and detect uncertain assignments that may need human review. - Singapore angle: "Classify HDB flats by type — is a jumbo flat a normal flat or a big flat? Under K-means you pick one. Under GMM it gets, say, 60% normal, 40% big, which is more honest." - "If beginners look confused": "Hard clustering is like yes/no. Soft clustering is like 70%/30%. Sometimes yes/no is fine. Sometimes you need to express uncertainty." - "If experts look bored": "Hard assignments are the argmax of soft assignments. K-means is the low-variance limit of EM on a spherical Gaussian mixture with equal mixing weights — they are

the same algorithm at different temperatures." **Transition**: "To learn soft assignments we need the EM algorithm. Two steps that alternate."

Slide 19: The EM Algorithm — E-Step

Time: ~4 min **Talking points**: - EM = Expectation-Maximisation. Two alternating steps that increase log-likelihood at every iteration. - E-step (Expectation): for each point, compute the responsibility r_nk = the probability that point n was generated by component k. This is Bayes' rule given current parameters. - Read the formula aloud slowly: r_nk = (mixing weight times Gaussian likelihood) / (sum over all components of the same thing). - Intuition: "Given my current guesses for the means and covariances, how likely is this point to have come from each cluster? Normalise those likelihoods so they sum to 1. Those are the responsibilities." - "If beginners look confused": "Forget the formula. The E-step is: for each customer, ask each cluster how likely it thinks that customer belongs to it, then turn those into percentages that add to 100." - "If experts look bored": "EM is a special case of variational inference where the variational posterior equals the true posterior. The E-step computes that posterior exactly because the model is conjugate." **Transition**: "Once every point has responsibilities, update the parameters."

Slide 20: The EM Algorithm — M-Step

Time: ~4 min **Talking points**: - M-step (Maximisation): update the parameters using the responsibilities as weights. New mean = weighted average of points. New covariance = weighted covariance. New mixing weight = average responsibility. - Read the formula: mu_k = sum over n of (r_nk times x_n) divided by sum over n of r_nk. - Compare to K-means: "In K-means, the centroid update is the mean of assigned points. In EM, the centroid update is the WEIGHTED mean of ALL points, weighted by responsibility. Every point contributes to every cluster, proportional to its responsibility." - "If beginners look confused": "The M-step says: now that I know how much each customer belongs to each cluster, I recompute the clusters using everyone's contribution, weighted by how much they belong." - "If experts look bored": "The M-step is the closed-form MLE of a weighted Gaussian — no gradient descent needed because the log-likelihood is concave in (mu, Sigma) given fixed responsibilities." **Transition**: "Alternate E and M. When do you stop?"

Slide 21: EM Convergence and Log-Likelihood

Time: ~3 min **Talking points**: - Log-likelihood is guaranteed non-decreasing at every EM iteration. Plot it — it should rise and plateau. - Convergence criterion: stop when the improvement between iterations drops below a tolerance (e.g. 1e-4). - Warning: EM converges to a LOCAL maximum of the log-likelihood. Run multiple starts and keep the best. - Visual: show the log-likelihood curve. It is monotone increasing — a straight non-decreasing line that curves into a plateau. If you see it drop, you have a bug. - "If beginners look confused": "The algorithm has a score it is trying to improve. Every round, it goes up or stays the same. When it stops going up, you stop." - "If experts look bored": "The monotonicity proof comes from Jensen's inequality applied to the expected complete-data log-likelihood. EM is a lower-bound maximisation, which is why it cannot decrease." **Transition**: "Apply this framework to Gaussian components and you get GMM."

Slide 22: GMM — EM Applied to Gaussians

Time: ~3 min **Talking points**: - GMM: assume the data was generated by a mixture of Gaussian distributions. Each Gaussian has a mean, a covariance, and a mixing weight. EM estimates all three. - Visual: two overlapping Gaussian blobs. K-means draws a hard boundary; GMM draws contour lines of probability. - Use GMM when

your clusters are elliptical (not round), when you want soft probabilities, or when you want a generative model you can sample from. - Model selection: use BIC or AIC to pick the number of components. - Singapore angle: "HDB resale transactions by price and area form elliptical clusters (prices and areas are correlated within flat types). K-means forces circles; GMM fits the actual shape." - "If beginners look confused": "GMM is K-means with ellipses instead of circles, and with percentages instead of hard labels." - "If experts look bored": "Full covariance GMMs have k * d * (d+1) / 2 parameters and overfit on small data. Use tied or diagonal covariance as a regulariser when d is large." **Transition**: "Mixture models are not just a clustering trick. The latest LLMs use them."

Slide 23: Mixture of Experts — Modern Application

Time: ~3 min **Talking points**: - Mixture of Experts (MoE): modern architecture where multiple "expert" networks specialise in different input regions. A gating network selects which experts to use for each input. - This is the same mathematical structure as GMM — mixing weights, component-specific parameters — but the components are neural networks instead of Gaussians. - Concrete: GPT-4 and many frontier LLMs are believed to use MoE. So is Mixtral (the open-source 8x7B model). Only a subset of experts activate per token, which makes inference cheap. - Payoff for today: "The EM template is not a toy. The soft-assignment pattern scales all the way to the models that power ChatGPT." - "If beginners look confused": "Think of a hospital with specialists. When you walk in, a receptionist looks at your symptoms and routes you to the right doctor. MoE is the same idea — a gate routes each input to the right expert network." - "If experts look bored": "Sparse MoE is trained with top-k gating and an auxiliary load-balancing loss — see Shazeer et al. 2017 and GShard — but the underlying probabilistic structure is still the mixture model you just derived." **Transition**: "Let us make all of this concrete in twenty lines of code."

Slide 24: Exercise 4.2 — EM from Scratch

Time: ~2 min **Talking points**: - Task: implement the EM algorithm in roughly twenty lines of polars-native Python on a 2D synthetic dataset with three Gaussians. Then compare with the sklearn GMM on real e-commerce data. - Visualisation requirement: plot the soft assignments (each point coloured by its responsibility vector) and the fitted ellipses. - Assessment: (1) your log-likelihood is monotone non-decreasing, (2) responsibilities sum to 1 for every point, (3) your 20-line implementation matches sklearn to within a tolerance. - "If beginners look confused": "The scaffolding gives you the structure. You fill in E-step and M-step. Twenty lines is a promise, not a challenge." **Transition**: "Soft clustering discovers groups. Dimensionality reduction discovers axes. Same theme — the algorithm discovers structure — different object."

Slide 25: 4.3 Dimensionality Reduction — Lesson Title

Time: ~1 min **Talking points**: - Subtitle: "Feature compression — discovering latent axes." - Framing: "Clustering compresses rows (many customers → a few segments). Dimensionality reduction compresses columns (many features → a few components). Both discover structure. Both are examples of the feature spectrum at work." - Preview: PCA, kernel PCA, t-SNE, UMAP, manifold learning, intrinsic dimension — ten slides, ~25 minutes. **Transition**: "Start with the workhorse of the other side: PCA."

Slide 26: PCA Step 1 — Decorrelate

Time: ~3 min **Talking points**: - PCA is a two-step process. Step 1: decorrelate. Rotate the axes so they align with the directions of greatest variance in the data. - Geometric intuition: imagine a cloud of points shaped like a

tilted ellipse. The original x and y axes are correlated. Rotate so the new axes align with the long and short sides of the ellipse. The new axes are uncorrelated. - The new axes are the principal components. They are eigenvectors of the covariance matrix, in decreasing order of eigenvalue. - “If beginners look confused”: “Imagine you take a photo of a banana from the wrong angle. PCA rotates the banana so you are looking along its length. Same banana, better axes.” - “If experts look bored”: “PCA is the eigendecomposition of the covariance matrix, or equivalently the SVD of the centred data matrix — the eigenvectors are the principal directions and the eigenvalues are the variances along those directions.” **Transition**: “Decorrelating is half the job. Reducing is the other half.”

Slide 27: PCA Step 2 — Reduce

Time: ~3 min **Talking points**: - Step 2: drop the lowest-variance components. Keep the top k that explain, say, 95% of the variance. - Scree plot: bar chart of variance explained per component. Look for the elbow, just like K-means. - Loadings: each principal component is a linear combination of the original features. Looking at the loadings tells you what the component “means” in business terms. - Singapore angle: “PCA on HDB features — floor area, storey, remaining lease, age, flat type — usually finds PC1 = size, PC2 = age/newness, PC3 = location premium. You did not design those axes; PCA discovered them.” - “If beginners look confused”: “PCA is like packing for a trip. You have fifty items but only one bag. PCA keeps the most important items (high variance) and leaves out the rest.” - “If experts look bored”: “Reduction is throwing away the low-eigenvalue eigenvectors. The reconstruction error equals the sum of the discarded eigenvalues — that is the Eckart-Young theorem.” **Transition**: “PCA has a twin: the singular value decomposition.”

Slide 28: PCA via SVD

Time: ~3 min **Talking points**: - X = U * Sigma * V^T, where X is your centred data matrix, U and V are orthogonal matrices, and Sigma is diagonal. - The columns of V are the principal components. The diagonal of Sigma gives the singular values — the square roots of the PCA eigenvalues. - Why use SVD instead of eigendecomposition of the covariance matrix? It is more numerically stable, and it works when n < d (more features than samples). - Preview the pivot: “Keep this factorisation in mind. At 4.7 we will factorise a different matrix — a user-item ratings matrix — using the same underlying idea, and that is when optimisation takes over.” - “If beginners look confused”: “SVD is a way to break any matrix into three simpler pieces. PCA is what you get when you apply SVD to your data and keep only the biggest pieces.” - “If experts look bored”: “SVD gives you the PCA basis without ever forming X^T X, which matters when d is in the millions and the covariance matrix does not fit in memory.” **Transition**: “How much information did we lose?”

Slide 29: Reconstruction Error

Time: ~2 min **Talking points**: - Reconstruction error: project the data onto the top-k components, then project back. Compute ||X - X_hat||^2. - That squared error equals the sum of the discarded eigenvalues. So if you keep 95% variance, the error is 5% of the total variance. - Use reconstruction error to decide k when you cannot eyeball a scree plot. - Connection to 4.7: “Matrix factorisation in 4.7 also minimises a reconstruction error. The objective $||R - U*V^T||2$ is literally the same thing. PCA minimises it under orthogonality; matrix factorisation minimises it under regularisation.” - “If beginners look confused”: “Reconstruction error is: if I throw away some information and then try to rebuild the data, how much am I off by? Less error means I threw away less important stuff.” **Transition**: “PCA is linear. What if the structure is curved?”

Slide 30: Kernel PCA

Time: ~2 min **Talking points**: - Linear PCA finds straight-line axes. Kernel PCA uses the kernel trick to find non-linear axes. - Common kernels: RBF (Gaussian), polynomial, sigmoid. Same kernels you may know from SVMs in M3. - Use kernel PCA when your data has clear non-linear structure (circles, spirals) that linear PCA cannot capture. - Cost: O(n^2) memory for the kernel matrix. Not practical beyond tens of thousands of points. - "If beginners look confused": "Linear PCA only draws straight lines. Kernel PCA can draw curves. If your data is curved, you need kernel PCA." - "If experts look bored": "Kernel PCA is eigen-analysis of the centred kernel matrix. It is equivalent to linear PCA in the feature space induced by the kernel map — which can be infinite-dimensional for RBF." **Transition**: "PCA is linear. Kernel PCA is kernelised. For visualisation specifically, t-SNE goes a different direction."

Slide 31: t-SNE — Visualisation Specialist

Time: ~3 min **Talking points**: - t-SNE = t-distributed Stochastic Neighbour Embedding. Projects high-dimensional data into 2D for visualisation. - Key parameter: perplexity. It controls how many neighbours each point considers. Typical values 5–50. Vary it — the plot can change significantly. - Strength: produces stunning, cluster-revealing 2D plots. Weakness: stochastic (different runs give different plots), no inverse transform (you cannot map new points), distances in the 2D plot are not meaningful. - CRITICAL caveat: "t-SNE is for visualisation only. Never use t-SNE as a feature extractor for a downstream model. Use PCA or UMAP for that." - "If beginners look confused": "t-SNE is the algorithm that makes those beautiful scatter plots where MNIST digits end up in clean separated blobs. It is for looking, not for modelling." - "If experts look bored": "t-SNE minimises the KL divergence between Gaussian similarities in input space and Student-t similarities in 2D space. The heavy-tailed Student-t fixes the crowding problem that affects SNE." **Transition**: "t-SNE has a modern replacement that fixes its weaknesses."

Slide 32: UMAP — The Modern Default

Time: ~3 min **Talking points**: - UMAP = Uniform Manifold Approximation and Projection. Faster than t-SNE, deterministic (with a random seed), preserves more global structure, and CAN be used for feature extraction. - Two key hyperparameters: n_neighbors (local vs global structure trade-off) and min_dist (how tightly points cluster). - "In 2026, UMAP is the default projection algorithm. Reach for it first. Only fall back to t-SNE if you need the specific aesthetic." - UMAP appears again at 4.6 (inside BERTopic) and 4.7 (visualising learned embeddings). It is a workhorse of this module. - "If beginners look confused": "UMAP makes the same kind of beautiful 2D plots as t-SNE, but faster and more reliable. When in doubt, use UMAP." - "If experts look bored": "UMAP is grounded in Riemannian geometry and algebraic topology — it constructs a fuzzy topological representation of the data and optimises a low-dimensional representation to match it. See McInnes, Healy, Melville 2018." **Transition**: "PCA, Kernel PCA, t-SNE, UMAP. There are a few more specialised methods you should know exist."

Slide 33: Manifold Learning — Reference Table

Time: ~2 min **Talking points**: - This slide is a reference, not a deep dive. Walk through the table: Isomap (preserves geodesic distances, good for unrolling), LLE (locally linear embedding, preserves local neighbourhoods), MDS (multidimensional scaling, preserves pairwise distances). - When to use each: Isomap if your data lies on a curved manifold (Swiss roll). LLE if local structure matters more than global. MDS if you have a distance matrix and want coordinates. - Honesty: "In practice most people go straight to UMAP. Know these exist so you are not caught off guard if a paper uses them." - "If beginners look confused": "Do not memorise this table. Just know it exists. If you ever need to unroll a Swiss roll, come back here." **Transition**: "How many dimensions do you actually need?"

Slide 34: Intrinsic Dimensionality

Time: ~2 min **Talking points**: - Intrinsic dimension: the minimum number of parameters needed to describe the data. Your data might have 100 columns but only 5 degrees of freedom. - Estimation: PCA scree plot, MLE estimators, correlation dimension. None are perfect. - Why it matters: “If the intrinsic dimension is low, you are wasting compute on high-dimensional methods. A linear model with 5 well-chosen features often beats a neural network with 100 raw features.” - Preview 4.8: “Neural network hidden layer size is a choice about how many latent dimensions you grant the model. Understanding intrinsic dimensionality gives you a principled starting point.” - “If beginners look confused”: “A photo has three colour channels and millions of pixels. But the faces in the photo can be described by maybe fifty numbers (nose shape, eye colour, jawline). Fifty is the intrinsic dimension. Millions is the measured dimension.” - “If experts look bored”: “Intrinsic dimension connects to the manifold hypothesis — that natural data lies on low-dimensional manifolds embedded in high-dimensional measurement spaces. This is the theoretical justification for representation learning.” **Transition**: “Let us put the three main methods to work.”

Slide 35: Exercise 4.3 — PCA, t-SNE and UMAP

Time: ~2 min **Talking points**: - Task: apply all three to the e-commerce dataset. Interpret the first three principal components via loadings. Produce a t-SNE and a UMAP visualisation. Demonstrate the reconstruction error trade-off as you vary k. - Assessment: scree plot with variance explained, loadings interpreted in business terms, hyperparameters varied and compared. - Pedagogy note: “The exercise deliberately makes you choose perplexity for t-SNE and n_neighbors for UMAP. There is no single right answer — the point is to build intuition for what each knob does.” - “If beginners look confused”: “Follow the notebook. The scaffolding makes the plots for you. Your job is to look at them and write what you see.” **Transition**: “Groups of similar points. Axes of variation. Now: points that belong to no group and no axis. Anomalies.”

Slide 36: 4.4 Anomaly Detection — Lesson Title

Time: ~1 min **Talking points**: - Subtitle: “Outlier discovery — finding what doesn’t belong.” - Framing: “Clustering finds groups of similar points. Anomaly detection finds points that belong to NO group. Two sides of the same coin.” - Preview: Z-score and IQR (from M2), Isolation Forest, LOF, score blending, production considerations — eight slides, ~20 minutes. **Transition**: “Start with the statistical methods you already know from M2.”

Slide 37: Statistical Outlier Detection

Time: ~2 min **Talking points**: - Z-score: z = (x - mean) / std. Flag anything with |z| > 3 (three-sigma rule). - IQR method: outlier if x < Q1 - 1.5*IQR or x > Q3 + 1.5*IQR. More robust because it does not assume normality. - Winsorisation: cap extreme values at a percentile instead of removing them. Useful when you cannot afford to lose rows. - Critical limitation: “These are univariate. They look at one column at a time. Real anomalies are often multivariate — a point can be normal in every single feature but anomalous in the combination.” - Singapore angle: “A $500,000 HDB resale transaction is normal. A 40-year-old flat is normal. A 40-year-old flat selling for $500,000 might be highly unusual depending on the type. Univariate tests miss this; multivariate methods catch it.” - “If beginners look confused”: “Z-score says: is this number far from average? IQR says: is this number far from the middle half? Simple, but only looks at one column at a time.” - “If experts look bored”: “Z-score breaks down on heavy-tailed distributions because the variance is inflated. Use the modified Z-score with median absolute deviation (MAD) for robustness: z_mod = 0.6745 * (x - median) / MAD.” **Transition**: “For multivariate anomalies, we need ML methods. Start with the most practical one.”

Slide 38: Isolation Forest

Time: ~3 min **Talking points**: - Idea: build random trees that isolate points by random splits. Anomalies are isolated in fewer splits. Normal points need many splits. - Intuition: “Anomalies are few and different. Few means they get isolated quickly by random splits. Different means they get separated early in the tree.” - Score: s(x, n) = 2^(-E[h(x)] / c(n)). Values near 1 are anomalies. - Why it is the most practical anomaly detector: fast (sub-linear training), handles high dimensions, makes no assumptions about distribution, scales to millions of points. - Singapore angle: “Fraudulent credit card transactions — anomalous dollar amounts combined with unusual merchants, unusual locations, unusual times. Isolation Forest catches them in real time.” - “If beginners look confused”: “Imagine playing twenty questions. A normal customer takes twenty questions to identify. A fraudster takes three. Isolation Forest measures how quickly it can identify each point.” - “If experts look bored”: “The expected path length for a point in a random tree under uniform splits is log(n). The Isolation Forest score normalises against this baseline. Extended Isolation Forest (EIF) improves on the axis-parallel split by using random hyperplanes.” **Transition**: “Isolation Forest is great for global anomalies. For local anomalies we need a density-based method.”

Slide 39: Local Outlier Factor (LOF)

Time: ~2 min **Talking points**: - Theory layer. LOF compares a point’s local density to its neighbours’ local densities. Ratio much greater than 1 means the point is in a sparse region relative to its neighbourhood. - When to use: data with regions of very different density, where Isolation Forest may miss locally anomalous points. - Cost: O(n^2) for naive implementation. Scale limits. - “If beginners look confused”: “Imagine a crowded street with one empty bench. LOF measures how empty the space around each person is compared to their neighbours. The person on the empty bench scores high even though everyone else on the street is close by.” - “If experts look bored”: “LOF uses reachability distance rather than raw distance to dampen density variation. See Breunig, Kriegel, Ng, Sander 2000 for the original formulation and its sensitivity to k.” **Transition**: “No single detector is best. Blend them.”

Slide 40: Score Blending — Combining Detectors

Time: ~2 min **Talking points**: - No single anomaly detector works everywhere. Z-score catches distributional outliers. Isolation Forest catches multivariate anomalies. LOF catches density anomalies. Blend them. - Recipe: normalise each detector’s scores to [0, 1], then combine with a simple average, weighted average, or max-vote. - This is ensemble thinking applied to unsupervised problems — the same idea as M3’s bagging and stacking, but without labels. - “If beginners look confused”: “If three detectives investigate a crime and all three flag the same suspect, you trust the verdict. Score blending is the same principle for anomaly detection.” - “If experts look bored”: “Unsupervised stacking is an active research area — LODA (Pevny 2016) builds many weak histogram-based detectors, and AutoOD picks detector combinations automatically.” **Transition**: “Kailash wraps this with one engine.”

Slide 41: Kailash Bridge — EnsembleEngine for Anomaly Detection

Time: ~3 min **Talking points**: - EnsembleEngine unifies `blend()`, `stack()`, `bag()`, and `boost()` behind one API. Today we use `blend()`. - Walk the code: instantiate EnsembleEngine, add each detector with a name, call `.blend()` with `method="weighted_average"`, filter by score threshold. - Key benefit: score normalisation is automatic. You do not have to calibrate thresholds by hand. - “If beginners look confused”: “EnsembleEngine is a kitchen mixer for detectors. You pour in ingredients, it blends them, you get a smooth output score.” - “If

experts look bored": "EnsembleEngine uses isotonic calibration when supervised validation data is available, otherwise rank normalisation. The same engine will reappear in M5 for ensemble inference." **Transition**: "Anomaly detection is not just a modelling exercise. It is a production monitoring tool."

Slide 42: Anomaly Detection in Production

Time: ~2 min **Talking points**: - Use cases: fraud detection, network intrusion, manufacturing quality control, data drift monitoring. - Production considerations: contamination rate (how many anomalies to expect), false positive cost vs false negative cost, online vs batch detection, explainability ("WHY is this an anomaly?"). - Connection to M3: "Anomaly detection on model inputs is how you detect data drift in production. If your inference traffic starts looking anomalous compared to your training distribution, your model is drifting." - Singapore angle: "A Singapore bank running Isolation Forest on card transactions cannot tolerate 5% false positives — every false positive is a blocked card and a call to the helpline. Threshold tuning is a business decision, not just a statistical one." - "If beginners look confused": "Detecting anomalies in the lab is easy. Deciding what to do about them in production is hard. That is the gap this slide is about." **Transition**: "Time for the exercise."

Slide 43: Exercise 4.4 — Financial Anomaly Detection

Time: ~2 min **Talking points**: - Task: financial transaction data. Apply Z-score, Isolation Forest, and LOF separately. Blend with EnsembleEngine. Identify which transactions are flagged by all three vs only one. Interpret which are true anomalies and which are false positives. - Assessment: multiple methods applied and compared, blended score improves over individual methods, business interpretation provided. - "If beginners look confused": "The exercise gives you the detectors. Your job is to compare them and decide which transactions you would actually investigate." **Transition**: "Groups, axes, outliers. Now: co-occurrence patterns. What items appear together?"

Slide 44: 4.5 Market Basket Analysis — Lesson Title

Time: ~1 min **Talking points**: - Subtitle: "Co-occurrence pattern discovery — finding what appears together." - Framing: "Clustering finds groups of similar POINTS. Association rules find groups of items that appear TOGETHER in transactions. A different kind of structure discovery." - Preview: Apriori, FP-Growth, metrics, applications — six slides, ~15 minutes. **Transition**: "The three metrics that power the entire field."

Slide 45: Association Rules — The Metrics

Time: ~3 min **Talking points**: - Support: how common is this itemset. supp(X) = count(X) / total transactions. If 20% of baskets contain bread, support of {bread} is 0.20. - Confidence: given X, probability of Y. conf(X → Y) = supp(X ∪ Y) / supp(X). "If you bought bread, what fraction of those baskets also had butter?" - Lift: is this more than chance? lift(X → Y) = conf(X → Y) / supp(Y). Lift > 1 = positive association. Lift = 1 = independent. Lift < 1 = negative. - Classic example: "Customers who buy diapers also buy beer" — the diaper-beer story is folklore, but lift of 2–3 on that rule is plausible and actionable. - Singapore angle: "NTUC FairPrice could discover that customers buying rice are 3x more likely to buy cooking oil in the same visit. That is lift > 1, and it drives shelf placement." - "If beginners look confused": "Support asks: is this common? Confidence asks: if I know one thing, how likely is the other? Lift asks: is this more than coincidence?" - "If experts look bored": "Lift is symmetric — lift(X → Y) equals lift(Y → X) — which is why confidence matters too. Use lift for interest and confidence for directionality." **Transition**: "Two algorithms to mine these rules at scale."

Slide 46: Apriori vs FP-Growth

Time: ~2 min **Talking points**: - Apriori: generate candidate itemsets, prune by minimum support, iterate. Classic, pedagogically simple, slow on large datasets because of the candidate generation step. - FP-Growth: compress the data into a frequent pattern tree (FP-tree), mine directly. Two passes over the data, no candidate generation, much faster. - In practice: FP-Growth for production, Apriori for teaching. - "If beginners look confused": "Apriori is a brute-force search. FP-Growth is a smart shortcut that builds an index first. Use the shortcut in production." - "If experts look bored": "FP-Growth's divide-and-conquer mining has time complexity linear in the number of frequent patterns, not the number of candidates, which is why it outperforms Apriori by orders of magnitude on sparse high-cardinality data." **Transition**: "Association rules are not a dead-end topic. They connect forward."

Slide 47: From Rules to Features

Time: ~3 min **Talking points**: - This is the design-level connection. Association rules discovered today can be used in two downstream ways. - First: rules as features. "Bought bread AND butter" becomes a binary column in your M3 supervised model. You take unsupervised discoveries and feed them into supervised learning. - Second — and this is the important one for the module's arc — co-occurrence patterns are exactly what collaborative filtering discovers in 4.7. Association rules find explicit co-occurrence. Collaborative filtering finds latent co-occurrence via matrix factorisation. Neural embeddings generalise both. - Feature Engineering Spectrum reminder: "Association rules are still hand-crafted in the sense that you pick the support threshold. Collaborative filtering lets optimisation decide. That is the pivot we are walking towards." - "If beginners look confused": "Rules are yes/no patterns. 4.7 will show you what happens when you replace yes/no with a number that a model learns. Same idea, softer." - "If experts look bored": "Word2Vec is literally a neural reparametrisation of co-occurrence matrix factorisation (Levy and Goldberg 2014). The thread from association rules to word embeddings to transformer attention is one continuous story." **Transition**: "Before we move on, note that this is not only a retail technique."

Slide 48: Applications Beyond Retail

Time: ~1 min **Talking points**: - Walk the table quickly: web analytics (page sequences), medicine (co-diagnoses and drug interactions), cybersecurity (attack pattern signatures), bioinformatics (gene co-expression), telecoms (service bundles), education (co-enrolment). - Point: "Anywhere you have 'transactions' — sets of co-occurring items — association rules apply." - "If beginners look confused": "The word 'basket' in 'market basket' is metaphorical. A basket of symptoms, a basket of clicks, a basket of genes — same algorithm." **Transition**: "Exercise."

Slide 49: Exercise 4.5 — Market Basket Analysis

Time: ~2 min **Talking points**: - Task: Singapore retail transactions. Implement Apriori and FP-Growth. Find top association rules. Compute support, confidence, lift. Interpret business meaning. Create binary features from the top rules and show they improve a classification model from M3. - Assessment: rules discovered with appropriate support threshold, business interpretation, rules-as-features improves the supervised model (connecting unsupervised discovery to supervised learning). - "If beginners look confused": "Start with a high support threshold so you get few rules. Lower it if you want more. The notebook guides you." **Transition**: "Items in a basket is one kind of unstructured data. Text is another. Same theme — discover structure."

Slide 50: 4.6 NLP — Text to Topics — Lesson Title

Time: ~1 min **Talking points**: - Subtitle: “Text feature discovery — extracting meaning from unstructured text.” - Framing: “In the last three lessons we found groups, axes, outliers, co-occurrences. Now we do the same for text. Text has no intrinsic columns — you must discover them. That is why NLP lives in Module 4.” - Preview: text representation, TF-IDF, word embeddings, LDA, NMF, BERTopic, coherence, sentiment — eight slides, ~25 minutes. **Transition**: “Before you can model text, you need to turn it into numbers.”

Slide 51: Text as Data — Representation

Time: ~3 min **Talking points**: - Text is not tabular. You must choose a representation. - Bag of words: count how often each word appears. Fast, interpretable, loses word order. - TF-IDF: weighted bag of words. Common words get low weight. Rare informative words get high weight. - Word embeddings: dense vectors that capture meaning. We will see them briefly later today — they derive from neural networks (4.8). - Preprocessing: lowercase, remove stopwords, stem or lemmatise, handle punctuation. The quality of your preprocessing often matters more than your algorithm choice. - “If beginners look confused”: “Imagine each document as a bag of Scrabble tiles. Count the tiles, and you have a representation. The algorithms we see today all start from that.” - “If experts look bored”: “Bag of words discards word order and therefore throws away syntax. Transformers (M5) recover word order via positional encoding — we will see why that matters next module.” **Transition**: “TF-IDF is the workhorse. Let us derive it.”

Slide 52: TF-IDF — Derivation

Time: ~4 min **Talking points**: - TF = term frequency: how often does term t appear in document d. Raw count, or count normalised by document length. - IDF = inverse document frequency: log(N / df(t)), where N is total documents and df(t) is the number of documents containing t. Rare terms get high IDF; common terms (like “the”) get IDF near zero. - TF-IDF = TF * IDF. High when a term appears often in this document but rarely across the corpus. That is the signature of a term that distinguishes this document from others. - Why it works: common words are not informative. Rare words that appear often in a specific document ARE informative. TF-IDF bakes that intuition into a formula. - Singapore angle: “Run TF-IDF on Straits Times articles. The word ‘Singapore’ has high TF but also very high DF, so its TF-IDF is near zero — it is not distinguishing. The word ‘dengue’ in a health article has high TF-IDF because it appears often in that article but rarely in others.” - “If beginners look confused”: “TF says the word is common HERE. IDF says the word is rare OVERALL. Multiply them: this word is common HERE but not everywhere. That is the definition of a keyword.” - “If experts look bored”: “TF-IDF is a crude approximation to the pointwise mutual information between term and document. BM25 refines it with saturation and length normalisation and is still the baseline for lexical retrieval in 2026.” **Transition**: “TF-IDF uses counts. Modern NLP uses embeddings.”

Slide 53: Word Embeddings — Tools, Not Derivation

Time: ~3 min **Talking points**: - Word embeddings are dense vectors that capture meaning. Similar words have similar vectors. Classic demonstration: king - man + woman ≈ queen. - Word2Vec (Mikolov 2013): CBOW predicts a word from its context, Skip-gram predicts the context from a word. Both learn vectors that represent “words that appear in similar contexts have similar meanings.” - GloVe: global co-occurrence statistics. FastText: subword embeddings, handles out-of-vocabulary words. - CRITICAL note: “I am telling you WHAT these do today. HOW they learn those vectors — that is the neural network training we will build from scratch in 4.8. Come back to this slide after 4.8 and the derivation will click.” - “If beginners look confused”: “Embeddings are the word version of what PCA did to your tabular data — many dimensions compressed into a few meaningful

ones. The difference is how they are learned.” - “If experts look bored”: “Word2Vec Skip-gram with negative sampling is implicitly factorising a shifted PMI matrix (Levy and Goldberg 2014). Embeddings are matrix factorisation in disguise, which is why they belong in this module right next to 4.7.” **Transition**: “TF-IDF and embeddings give you features. How do you discover topics?”

Slide 54: LDA — Latent Dirichlet Allocation

Time: ~3 min **Talking points**: - LDA is a generative probabilistic model. Assumption: each document is a mixture of topics, and each topic is a distribution over words. - To “generate” a document: pick a topic mix, then for each word pick a topic from the mix and a word from that topic. - Given real documents, we invert the generative story and learn the topic-word distributions and document-topic distributions. - Output: each topic is a list of its top words, and each document has a percentage breakdown across topics. - Connection to 4.2: “LDA is conceptually a mixture model — soft assignment of words to topics, just like GMM’s soft assignment of points to Gaussians. The EM template from 4.2 applies here with a Dirichlet prior.” - “If beginners look confused”: “A topic is like a theme. LDA says every document is a mix of themes, and every theme has favourite words. Given a pile of articles, LDA finds the themes for you.” - “If experts look bored”: “Vanilla LDA is fit with collapsed Gibbs sampling or variational inference. Online LDA (Hoffman, Blei, Bach 2010) made it scalable to web-size corpora.” **Transition**: “LDA is the classical answer. Two modern alternatives.”

Slide 55: NMF and BERTopic

Time: ~3 min **Talking points**: - NMF (Non-negative Matrix Factorisation): factorises the TF-IDF matrix into W * H with non-negativity constraints. W is documents by topics, H is topics by words. Same spirit as PCA but with a non-negativity constraint that forces interpretable parts-based decomposition. - Echo of the PCA / matrix factorisation / pivot thread: “NMF is matrix factorisation of the text. It is another point on the same spectrum that is about to pivot at 4.7.” - BERTopic: modern transformer-based topic modelling. Pipeline: BERT embeddings → UMAP → HDBSCAN → class-based TF-IDF. It reuses three techniques we have already met today (UMAP, HDBSCAN) plus a transformer from M5. - BERTopic is usually the best quality out of the box but depends on a pretrained language model. - “If beginners look confused”: “LDA is the classical method. NMF is the linear algebra method. BERTopic is the modern method that stacks transformers, UMAP, and HDBSCAN. When in doubt, try BERTopic first.” - “If experts look bored”: “Class-based TF-IDF (c-TF-IDF) treats each cluster as a single pseudo-document and computes TF-IDF across clusters, which gives interpretable topic labels for free. That is BERTopic’s key engineering insight.” **Transition**: “How do you know a topic model is any good?”

Slide 56: Topic Coherence — Evaluating Quality

Time: ~2 min **Talking points**: - Coherence metrics measure how semantically related the top words in a topic are. - NPMI (Normalised Pointwise Mutual Information): are these words statistically associated with each other? - UMass: are these words frequently co-occurring in the corpus? - Higher coherence usually correlates with human interpretability. Not always — human judgement is still the final check. - “If beginners look confused”: “Coherence asks: do the words in a topic actually hang together? If the top words in a topic are ‘bank’, ‘money’, ‘loan’, that is coherent. If they are ‘bank’, ‘tree’, ‘phone’, that is not.” - “If experts look bored”: “Coherence metrics correlate imperfectly with human ratings. Always pair coherence with manual inspection on a sample. See Lau, Newman, Baldwin 2014 for empirical comparisons.” **Transition**: “One application-level slide before the exercise.”

Slide 57: Sentiment Analysis — Text Classification

Time: ~2 min **Talking points**: - Sentiment analysis: classify text as positive, negative, or neutral. A supervised problem technically, but shown here because it is the most common downstream application of the text features we just discovered. - Classical pipeline: TF-IDF features → logistic regression or linear SVM. Simple, fast, surprisingly competitive. - Modern pipeline: fine-tuned transformer. More accurate but more expensive. Covered in M5. - Singapore angle: "Classify Singapore customer reviews on food delivery apps. Detect which menu items drive satisfaction vs complaints. Simple TF-IDF + logistic regression hits 85% accuracy on English-Singlish review data." - "If beginners look confused": "Sentiment is just text classification with labels 'positive' and 'negative'. You already know how classification works from M3." **Transition**: "Exercise."

Slide 58: Exercise 4.6 — Topic Modelling and Sentiment

Time: ~2 min **Talking points**: - Task: Singapore news articles. Extract topics with TF-IDF + NMF, LDA, and BERTopic. Compare topic quality with NPMI. Classify sentiment on customer reviews with TF-IDF + logistic regression. - Assessment: multiple topic methods compared, coherence metrics computed, topics interpreted with human-readable labels. - "If beginners look confused": "The notebook does most of the preprocessing for you. Your job is to run three topic models, look at the top words, and label the topics." **Transition**: "Take a five-minute break. When we come back, we reach THE PIVOT of the entire module."

[BREAK — 5 min]

Slide 59: 4.7 Recommender Systems — Lesson Title

Time: ~1 min **Talking points**: - Subtitle: "THE PIVOT — optimisation drives feature discovery." - Frame the stakes: "What you see in this lesson is the single most important concept in the entire curriculum. Everything before this lesson discovered features with fixed algorithms. From this lesson forward, features are discovered by OPTIMISATION." - Preview: content-based vs collaborative filtering, user-based vs item-based, matrix factorisation, ALS, implicit feedback, THE PIVOT slide, embedding visualisation — nine slides, ~25 minutes. **Transition**: "Start with the two main families of recommenders."

Slide 60: Content-Based vs Collaborative Filtering

Time: ~3 min **Talking points**: - Content-based: recommend items similar to what the user already liked, based on item features. You liked this action movie → here is another action movie. - Collaborative filtering: recommend items that similar users liked. People who liked this also liked that. No item features required. - Strengths and weaknesses: content-based needs good item features but handles new items (cold start). Collaborative filtering needs no features but struggles with new items and users. - In practice everyone uses a hybrid. - Singapore angle: "Shopee's recommendation system blends both — content-based for new listings, collaborative for the product tail." - "If beginners look confused": "Content-based says: you liked a horror movie, so here is another horror movie. Collaborative says: people who liked your movies also liked this one, so try it. Both are valid." - "If experts look bored": "Content-based models often reduce to nearest-neighbour search over item embeddings. Collaborative filtering is where matrix factorisation lives, and it is the interesting bit for today's pivot." **Transition**: "Within collaborative filtering there are two subfamilies."

Slide 61: User-Based vs Item-Based CF

Time: ~2 min **Talking points**: - User-based: find similar users, recommend their items. Problem: new user has no ratings, so no neighbours. Cold start. - Item-based: find similar items, recommend to users who liked similar

items. More stable because item similarity changes slowly. - Amazon popularised item-based CF in the early 2000s because it scaled better than user-based. - “If beginners look confused”: “User-based asks: who is like me? Item-based asks: what is like what I already bought? Both work, but item-based is more reliable because items do not change mood.” - “If experts look bored”: “Item-item CF is memory-based and has O(n_items^2) similarity computation. Matrix factorisation is the model-based alternative that scales better — coming up next.” **Transition**: “Memory-based CF has limits. Now the breakthrough.”

Slide 62: Matrix Factorisation — The Core Idea

Time: ~4 min **Talking points**: - Setup: you have a user-item rating matrix R. Rows are users, columns are items, cells are ratings. Most cells are empty — that is the recommendation problem. - Idea: factorise R into two smaller matrices U (user embeddings) and V (item embeddings), such that R ≈ U * V^T. - U is n_users by k. V is n_items by k. k is the number of latent dimensions (say, 50 or 100). Each user is a k-dim vector; each item is a k-dim vector. A rating is the dot product of the two. - Draw the factorisation on the whiteboard if possible. “This is PCA with a twist — PCA factorised a full matrix to decorrelate features. Matrix factorisation factorises a SPARSE matrix to FILL IN the missing ratings.” - This is the same formula from 4.3: X = U * Sigma * V^T. We lose Sigma (absorbed into U and V) but keep the factorisation. Deep connection between dimensionality reduction and collaborative filtering. - “If beginners look confused”: “Imagine that every user has a personality vector with 50 numbers (likes drama, likes action, likes old films, etc). Every movie has a profile vector with the same 50 numbers. A rating is how well the user’s personality matches the movie’s profile. Matrix factorisation learns both sets of vectors at once from the ratings alone.” - “If experts look bored”: “This is the Netflix Prize architecture (Koren, Bell, Volinsky 2009). The breakthrough was realising that a low-rank factorisation of the ratings matrix outperforms every neighbour-based method by a wide margin — and the factors are interpretable embeddings.” **Transition**: “How do you actually learn U and V?”

Slide 63: ALS — Alternating Least Squares

Time: ~3 min **Talking points**: - The objective: minimise the sum over observed ratings of (r_ui - u_u^T * v_i)^2, plus a regularisation term on the norms of U and V. - This objective is NOT convex in (U, V) jointly. But if you fix U, it IS convex in V. And if you fix V, it IS convex in U. That is the ALS trick. - Algorithm: fix U, solve for V analytically (least squares). Fix V, solve for U analytically. Alternate. Convergence is fast and deterministic. - Why it matters: this is the first time in the curriculum that we are running OPTIMISATION to discover features. Not a closed-form eigendecomposition. Not a fixed clustering algorithm. An actual loss function we minimise. - Singapore angle: “Build ALS on Shopee-style transaction data: users by items, click-through rates in the cells. Learn 64-dimensional embeddings for each item. Similar items end up near each other in the 64-dim space — even though nobody labelled them by category.” - “If beginners look confused”: “ALS is: you have two unknowns. Fix one, solve for the other. Then fix the other, solve for the first. Repeat. Each step is simple, and together they solve a hard problem.” - “If experts look bored”: “ALS is a block coordinate descent on a bi-convex objective. It has closed-form updates because each block is a ridge regression. The alternative is SGD with implicit-feedback weighting, which powers YouTube’s recommender.” **Transition**: “Real recommenders rarely have explicit ratings. They have clicks and views.”

Slide 64: Implicit Feedback and Hybrid Systems

Time: ~2 min **Talking points**: - Explicit feedback: users rate items 1-5. Honest but scarce. - Implicit feedback: users click, view, dwell, purchase. Abundant but noisy — a click is not necessarily a like. - Implicit ALS: weight observed interactions by confidence, treat missing values as weak negatives. Powered Spotify’s early Discover Weekly. - Hybrid systems: combine content-based (for new items and features) with collaborative (for the tail). Almost every production system is a hybrid. - “If beginners look confused”: “If you scroll past a video, that is

almost a ‘no’. If you watch the whole thing, that is almost a ‘yes’. Implicit feedback is counting those signals and treating them as approximate ratings.” - “If experts look bored”: “See Hu, Koren, Volinsky 2008 for the implicit ALS formulation. The key is the confidence-weighted MSE: c_ui * (p_ui - u_u^T v_i)^2 with c_ui increasing in observed interaction strength.” **Transition**: “Now — the slide I have been pointing at all day.”

Slide 65: THE PIVOT — Optimisation Drives Feature Discovery

Time: ~4 min **Talking points**: - SLOW DOWN. This is the most important slide of the module after the Feature Engineering Spectrum. - Read the headline aloud verbatim: “Matrix factorisation learns user and item EMBEDDINGS by minimising reconstruction error.” - Then: “This is the first time you have seen OPTIMISATION DRIVE FEATURE DISCOVERY.” - Walk the spectrum diagram on the slide. 4.7 matrix factorisation on the left. 4.8 neural networks on the right. The arrow says: same idea, generalised. - Unpack the generalisation: “In 4.7 you factorise R into U * V^T. The embeddings U and V are LINEAR functions of the observed ratings — a sum of products. In 4.8 you build a neural network. Hidden layer activations a = f(Wx + b) are NON-LINEAR functions of the input. They are still embeddings. They are still learned by minimising a loss. The difference is the activation function.” - Repeat the key line for emphasis: “Hidden layer activations ARE embeddings, learned by minimising a loss function.” - Feature Engineering Spectrum reminder: “Before today, features were designed (M1-M3) or discovered by fixed algorithms (4.1-4.6). Here at 4.7, optimisation takes over. At 4.8 that same optimisation gains non-linearity and the modern AI story begins.” - “If beginners look confused”: “You have seen this idea once already today and did not know it was the same idea. When you ran K-means, you minimised within-cluster sum of squares. That was optimisation. Matrix factorisation is the same spirit, except the thing you are optimising is an entire embedding space for every user and every item. And that is exactly what a neural network hidden layer is.” - “If experts look bored”: “This is the theoretical bridge from classical ML to deep learning. It is also the technical reason transformers work — attention is a learned factorisation of a token co-occurrence structure. The path from Netflix Prize to GPT-4 runs through this slide.” **Transition**: “Let me show you what these embeddings look like.”

Slide 66: Visualising Learned Embeddings

Time: ~3 min **Talking points**: - Take the 64-dimensional item embeddings from ALS. Project to 2D with UMAP (from 4.3 — reusing the technique). - Result: similar items cluster together. Comedy movies form a region. Action movies form a region. Romance movies form another. - Key insight: “Nobody told ALS about movie genres. It discovered them from the rating patterns alone. That is what ‘discovered by optimisation’ looks like.” - Same pattern works for users: similar users cluster. You can identify taste profiles geometrically. - Preview 4.8: “Now imagine the same visualisation for a neural network’s hidden layer on the same dataset. You would see the same thing — because a neural network hidden layer is doing the same job, non-linearly.” - “If beginners look confused”: “The embeddings are coordinates the algorithm invented. When you look at the map, you can SEE what it learned — because similar things end up in the same neighbourhood.” - “If experts look bored”: “This is the same visualisation technique Word2Vec used in the original paper to show king-queen-man-woman analogies. The embedding geometry is the feature. See Mikolov, Yih, Zweig 2013 for the linguistic regularities story.” **Transition**: “Your turn.”

Slide 67: Exercise 4.7 — Recommender Systems

Time: ~2 min **Talking points**: - Task: Singapore retail transaction data. Build user-based CF, item-based CF, and matrix factorisation with ALS. Compare recommendation quality. Visualise learned embeddings with UMAP. Write a short paragraph articulating THE PIVOT in your own words. - Assessment: all three approaches implemented and compared, embeddings visualised with meaningful clusters, the pivot concept articulated in writing. - Writing requirement is deliberate: “If you cannot explain the pivot in your own words, you have not

yet understood what 4.7 is about. The paragraph is the check." - "If beginners look confused": "The scaffolding gives you ALS. Your job is to run it, look at the embeddings, and explain what you see." **Transition**: "The pivot generalises. Time to build a neural network from scratch."

Slide 68: 4.8 DL Foundations — Lesson Title

Time: ~1 min **Talking points**: - Subtitle: "Neural Networks, Backpropagation and the Training Toolkit." - Read the bridge line: "In 4.7, matrix factorisation learned embeddings by minimising reconstruction error. A neural network does the same thing — hidden layer activations ARE embeddings, learned by minimising a loss function. The difference is non-linearity." - Set expectations for density: "This is the biggest lesson in the module. Seventeen slides. Expect to move quickly. Everything you have seen today is a subset of what happens inside a neural network." **Transition**: "Visualise the bridge one more time."

Slide 69: The Bridge — From Matrix Factorisation to Neural Networks

Time: ~2 min **Talking points**: - Show the spectrum one more time. Manual features on the left, USML in the middle, deep learning on the right — highlighted, because we have arrived. - Key line: "Hidden layers = automated feature engineering WITH gradient-based error feedback." - Unpack the two halves: "Automated feature engineering — that is what all of 4.1 through 4.7 was doing. Gradient-based error feedback — that is the part we are about to add. Put them together and you get deep learning." - "If beginners look confused": "You already know the first half (features are discovered). Today you add the second half (the error tells the features how to update)." - "If experts look bored": "The unifying view is: hidden layers compute learned nonlinear features, and the training loop uses gradient descent to tune those features for the downstream task. That is the Bengio-Lecun-Hinton representation learning thesis from 2012." **Transition**: "Start with the architecture."

Slide 70: Neural Network Architecture

Time: ~3 min **Talking points**: - Input layer: one node per feature. Hidden layers: one or more layers of nodes between input and output. Output layer: one node for regression, N nodes for N-class classification. - Each node in a layer is connected to every node in the next layer — "fully connected" layers. - Each connection has a weight. Each node has a bias. - The full network is a giant parametric function: input → hidden layer 1 → hidden layer 2 → … → output. - Draw a 2-3-1 network on the whiteboard if possible. Two inputs, three hidden nodes, one output. Count the parameters: 6 weights for the first layer + 3 biases + 3 weights for the second layer + 1 bias = 13 parameters. - "If beginners look confused": "Each layer is a grid of light bulbs. The previous layer sends voltages. Each bulb mixes the voltages with its own knobs (weights) and lights up. The pattern of lights is what the next layer sees." - "If experts look bored": "Modern architectures replace full connectivity with structured sparsity (convolutions for vision, attention for sequences). But the fully connected network is the theoretical starting point — universal approximation lives here." **Transition**: "How does information flow through the network?"

Slide 71: Forward Pass

Time: ~3 min **Talking points**: - Forward pass: feed input into the network, compute activations layer by layer, produce prediction. - At each node: z = W * x + b (weighted sum), then a = f(z) (activation function). Pass a to the next layer as input. - Three things to track: inputs x, weighted sums z, activations a. You will need all three

for backprop later. - Walk through a tiny concrete example with real numbers if possible. Two inputs, three hidden nodes, one output. Plug in numbers. Show z and a at each layer. - “If beginners look confused”: “Forward pass is: plug in your inputs, multiply by weights, add up, squish through an activation, pass along. Repeat for each layer. At the end you get a prediction.” - “If experts look bored”: “Forward pass is matrix multiplication followed by element-wise activation. On modern hardware, the whole forward pass for a million-row batch is a handful of GPU kernels — see the fused kernels in cuBLAS and cuDNN.” **Transition**: “A neural network with zero hidden layers is something you already know.”

Slide 72: Linear Regression as a Neural Network

Time: ~3 min **Talking points**: - Take a neural network with one linear output node and no hidden layers. y_hat = W * x + b. That is linear regression. - Same model you used in M2 and M3. Same MSE loss. Same gradient descent. - Now add a hidden layer with a non-linear activation: y_hat = W_2 * relu(W_1 * x + b_1) + b_2. That is no longer linear. That is a 2-layer MLP, and 2 hidden layers can approximate any continuous function (universal approximation theorem). - Key line: “Adding a hidden layer lets the model WRITE ITS OWN PARAMETRIC FUNCTION. You do not pick the features. The hidden layer discovers them by optimisation.” - Singapore angle: “Predict HDB resale price from area, storey, age. Linear regression: assume price is a linear combination of those three. 2-layer MLP: the hidden layer can discover ‘size premium for high floors’ and ‘age penalty for lease decay’ automatically as non-linear features.” - “If beginners look confused”: “A neural network with no hidden layer is linear regression. Add one layer, and it can bend. That is the whole difference.” - “If experts look bored”: “The universal approximation theorem (Cybenko 1989, Hornik 1991) says two-layer networks with enough hidden units can approximate any continuous function on a compact domain. Depth — which we exploit in M5 — gives exponential parameter efficiency compared to width.” **Transition**: “How do you train it?”

Slide 73: Loss Functions and Gradient Descent

Time: ~4 min **Talking points**: - Loss function: measures how wrong the prediction is. For regression, MSE = (1/2) * (y - y_hat)^2. For classification, cross-entropy. Loss is a number you want to minimise. - Gradient descent: compute the gradient of the loss with respect to each weight. Update each weight by subtracting learning_rate * gradient. Repeat until the loss stops decreasing. - Walk through a one-weight example. w_new = w_old - 0.01 * gradient. Show that if the gradient is positive, the weight decreases. If negative, it increases. Always towards lower loss. - Singapore angle: “HDB example. You predict a price, you see the real price. The error tells you which way to nudge the weights so next time the prediction is closer. Thousands of nudges later, the model has learned.” - “If beginners look confused”: “Gradient descent is: take a small step downhill, look at where you are, take another small step downhill. Repeat until you cannot go any lower. The hill is the loss function.” - “If experts look bored”: “Stochastic gradient descent estimates the full gradient with a mini-batch, which is why it converges in wall-clock time despite being biased. The modern recipe is SGD + momentum or Adam with warmup — coming up in a few slides.” **Transition**: “Gradient descent needs gradients. How do you compute them through many layers?”

Slide 74: Backpropagation — Chain Rule Through Layers

Time: ~4 min **Talking points**: - Backpropagation is the chain rule from calculus, applied layer by layer, from the output back to the input. - Intuition: you know the loss at the output. You want to know how each weight in each layer contributed to that loss. The chain rule gives you a clean way to propagate the error signal backwards. - Write the chain on the whiteboard: dL/dw_1 = dL/da_2 * da_2/dz_2 * dz_2/da_1 * da_1/dz_1 * dz_1/dw_1. That is one partial derivative for each layer you pass through. - Why it works: each layer’s gradient depends only on the previous layer’s activation and the next layer’s incoming gradient. It is a local computation, which is why

it scales to networks with billions of parameters. - “If beginners look confused”: “Backprop is a rumour game in reverse. The output layer knows the error. It whispers to the layer before it: ‘you contributed this much.’ That layer whispers to the layer before IT, and so on back to the input. Each layer learns exactly how much to update.” - “If experts look bored”: “Backprop is reverse-mode automatic differentiation with a specific topological order. Modern frameworks (PyTorch, JAX) implement autodiff via computational graphs, but the underlying algorithm is the one you will implement by hand in today’s exercise.” **Transition**: “So what are hidden layers DOING?”

Slide 75: Hidden Layers = Automated Feature Engineering

Time: ~4 min **Talking points**: - THIS is the lesson’s version of the pivot slide. Say it clearly: “Hidden layers are automated feature engineering with error feedback.” - Unpack the claim: - Hidden node values are LEARNED functions of the input. They are features the network invented. - Those features are updated by BACKPROP so they MINIMISE the loss. The optimisation drives feature discovery. - Two or more hidden layers can represent ANY non-linear function (universal approximation). - Hidden activations are embeddings — exactly like the embeddings from 4.7, but non-linear. - The “unsupervised meets supervised” line: “Hidden layers perform unsupervised feature discovery while the output layer performs supervised prediction. They train TOGETHER. The features are shaped by the task.” - Come back to the Feature Engineering Spectrum one last time. “Left side: human picks features. Middle: algorithm discovers features. Right side: the network learns features shaped by the task. Everything you have learned today lives on this line, and this slide is where it ends.” - “If beginners look confused”: “This is the single most important idea in modern AI. A neural network is not magic. It is feature engineering that happens inside the model itself, guided by the error signal. That is why it works so well — the features are exactly the ones the task needs.” - “If experts look bored”: “Representation learning (Bengio, Courville, Vincent 2013) formalised this idea. Deep networks learn hierarchies — low layers learn edges, mid layers learn parts, top layers learn concepts. The feature engineering you would have done by hand is now emergent from gradient descent.” **Transition**: “The rest of the lesson is the training toolkit — the practical machinery that makes all of this actually work.”

Slide 76: Activation Functions

Time: ~3 min **Talking points**: - Activation functions introduce non-linearity. Without them, stacked linear layers collapse into a single linear layer. - ReLU (Rectified Linear Unit): f(x) = max(0, x). The default for hidden layers. Fast, solves the vanishing gradient problem for positive inputs, but can “die” (output always 0) if inputs are negative. - Leaky ReLU, PReLU, ELU, GELU, Swish: variants that fix the dying ReLU problem. GELU is the default for transformers in M5. - Sigmoid: f(x) = 1 / (1 + exp(-x)). Used for binary classification output. Squashes to [0, 1]. Causes vanishing gradients in hidden layers — avoid for deep networks. - Tanh: like sigmoid but output is [-1, 1]. Used in older architectures. - Softmax: for multi-class output. Turns a vector of logits into a probability distribution. - Quick rule: ReLU or GELU in hidden layers, sigmoid for binary output, softmax for multi-class. - “If beginners look confused”: “Activation functions are the ‘squish’ at each node. Without them a neural network is just fancy linear regression. The squish is what lets it bend.” - “If experts look bored”: “GELU is ReLU with a smooth transition region, weighted by the CDF of a Gaussian — see Hendrycks and Gimpel 2016. Swish and SiLU are essentially equivalent.” **Transition**: “Two regularisation techniques you will reach for every time.”

Slide 77: Dropout and Batch Normalisation

Time: ~3 min **Talking points**: - Dropout: randomly zero out a fraction of neurons during training. Forces the network not to rely on any single neuron. Prevents co-adaptation. Typical dropout rate 0.1 to 0.5. CRITICAL: turn off at inference time. - Batch normalisation: normalise each layer’s inputs to zero mean and unit variance, per mini-batch. Stabilises training. Enables higher learning rates. Reduces sensitivity to weight initialisation. -

Batch norm is a secret weapon: you can almost always train faster with it on. - “If beginners look confused”: “Dropout is: randomly turn off some lights during training so the rest of the network learns to work without them. Batch norm is: squeeze the numbers at each layer into a standard range so training is less chaotic.” - “If experts look bored”: “BatchNorm has a known issue with small batch sizes and distributed training. LayerNorm (Ba, Kiros, Hinton 2016) fixes that and is the default for transformers. We will see layer norm in M5.” **Transition**: “Before you train you have to initialise. Wrong initialisation kills training.”

Slide 78: Weight Initialisation

Time: ~2 min **Talking points**: - Why weights must be random: if all weights start at zero, all neurons in a layer learn the same thing (symmetry). Random initialisation breaks symmetry. - Xavier / Glorot: draws weights from a distribution scaled by the fan-in and fan-out. Designed for sigmoid / tanh activations. - Kaiming / He: similar but designed for ReLU. Default in modern frameworks. - Quick rule: use Kaiming for ReLU networks, Xavier for tanh/sigmoid. Almost never think about this again — the framework default is usually right. - “If beginners look confused”: “If you start all the neurons identical, they all learn the same thing. Random init makes them different from the start so they can specialise.” - “If experts look bored”: “Kaiming init preserves the variance of activations through ReLU layers, which prevents exploding or vanishing signals in deep networks. See He, Zhang, Ren, Sun 2015.” **Transition**: “Now the optimiser — the thing that actually does the updating.”

Slide 79: Optimisers — From SGD to Adam

Time: ~3 min **Talking points**: - SGD (Stochastic Gradient Descent): the basic optimiser. Update = -learning_rate * gradient. Slow but simple. - SGD + momentum: accumulate a running average of gradients. Helps skate through flat regions and accelerate down consistent slopes. - RMSProp: adapts per-parameter learning rates using a moving average of squared gradients. - Adam: momentum + RMSProp together. Default optimiser in 2026 for most tasks. Easy to use, usually works. - AdamW: Adam with decoupled weight decay. Preferred over plain Adam for transformers. - Quick rule: start with AdamW, fall back to SGD + momentum if AdamW overfits. - “If beginners look confused”: “The optimiser is the thing that actually walks downhill. SGD walks carefully. Adam walks confidently. In practice you use Adam.” - “If experts look bored”: “Adam’s warmup behaviour is one of the hidden subtleties — the first few steps use biased moment estimates, which is why warmup schedules exist. See Kingma and Ba 2014, Loshchilov and Hutter 2017 for AdamW.” **Transition**: “The loss function tells the optimiser what to minimise. Pick the right one.”

Slide 80: Loss Functions Taxonomy

Time: ~3 min **Talking points**: - Walk the table. MSE for regression. MAE for robust regression. Cross-entropy for classification. Binary cross-entropy for binary. Focal loss for imbalanced classes. Contrastive loss for similarity learning. Triplet loss for metric learning. KL divergence for distribution matching. Reconstruction loss for autoencoders. - The loss function encodes what you care about. Get it right and the network learns the right thing. Get it wrong and the network optimises the wrong objective. - MSE vs MAE: MSE punishes outliers more. Use MAE when outliers are legitimate and should not dominate. - Cross-entropy vs MSE for classification: always cross-entropy. MSE with sigmoid outputs leads to vanishing gradients. - “If beginners look confused”: “The loss function is the scoreboard. Pick the wrong scoreboard and the network cheats to win the wrong game. Regression → MSE. Classification → cross-entropy. That is 90% of what you need.” - “If experts look bored”: “Focal loss (Lin et al. 2017) adds a (1-p)^gamma modulating factor that down-weights easy examples — essential for dense detection. Triplet and contrastive losses underpin metric learning and are the basis of modern self-supervised representation learning.” **Transition**: “One more piece — how the learning rate changes over time.”

Slide 81: Learning Rate Schedules and Early Stopping

Time: ~3 min **Talking points**: - Fixed learning rate is rarely optimal. Schedules decay the learning rate over time so you can make big moves early and small moves late. - Step decay: drop the rate at fixed epochs. Cosine annealing: smooth decay. Warmup + cosine: ramp up over first few epochs, then anneal. One-cycle policy: ramp up, then down. ReduceLROnPlateau: drop when validation loss plateaus. - Early stopping: watch validation loss. Stop when it stops improving for N epochs (patience). Prevents overfitting. Keep the best checkpoint. - Gradient clipping (brief): if gradients explode, cap them at a max norm. Required for RNNs and sometimes transformers. - "If beginners look confused": "Learning rate schedules are the training equivalent of slowing down as you get close to the destination. Early stopping is the equivalent of knowing when to park." - "If experts look bored": "One-cycle training (Smith 2018) is a fast-converging alternative that treats the learning rate as a parameter to be super-convergence-tuned. The training toolkit is full of small tricks like this — the exercise asks you to turn each one on and off to see the effect." **Transition**: "Two notes on the output layer."

Slide 82: Regression vs Classification Output

Time: ~2 min **Talking points**: - Regression: one output node with a linear activation. No softmax, no sigmoid. MSE loss. - Binary classification: one output node with sigmoid activation. Binary cross-entropy loss. - Multi-class classification: N output nodes (one per class) with softmax activation. Categorical cross-entropy loss. - The choice is mechanical. Match the output layer and the loss to the task. - "If beginners look confused": "Continuous number → linear output + MSE. Yes/no → sigmoid + BCE. Pick-one-of-many → softmax + CE. Match these three and you are done." - "If experts look bored": "Softmax + cross-entropy is the log-likelihood of a categorical distribution. Sigmoid + BCE is the log-likelihood of a Bernoulli. Both are maximum-likelihood training under different parametric output distributions." **Transition**: "Kailash wraps all of this."

Slide 83: Kailash Bridge — OnnxBridge for Neural Networks

Time: ~3 min **Talking points**: - OnnxBridge is the Kailash engine for training, exporting, and serving neural networks. You define a model, OnnxBridge handles the training loop, checkpointing, ONNX export, and inference serving. - Show the code block. Point out that OnnxBridge uses the same ExperimentTracker and ModelRegistry you used in M3. Unified toolchain. - Philosophy: "You are building your first neural network from scratch in the exercise so you know what the engine is doing. In production, OnnxBridge saves you hours of boilerplate." - ONNX note: the export format makes your trained model portable across PyTorch, TensorFlow, and inference runtimes. That matters for M5 and M6 production deployment. - "If beginners look confused": "OnnxBridge is the wrapper that turns your trained network into a production-ready service. You focus on the model; it handles the rest." - "If experts look bored": "ONNX standardises the computation graph and operator set, which is why a PyTorch model can run on an inference server optimised for different hardware. OnnxBridge is the Kailash glue." **Transition**: "Now the centrepiece exercise."

Slide 84: Exercise 4.8 — Neural Network from Scratch

Time: ~3 min **Talking points**: - This is the biggest exercise in the module. Expect 45-60 minutes. Do not rush. - Task part 1: build a 3-layer neural network from scratch in polars + numpy for HDB price prediction. Forward pass, MSE loss, backprop, gradient descent. No framework. - Task part 2: after the hand-built version works, add dropout, batch norm, Adam optimiser, and a learning rate schedule. Plot training curves with and without each technique. - Task part 3: write a short paragraph explaining "hidden layers perform unsupervised feature discovery while the output layer performs supervised prediction." - Assessment: from-scratch implementation converges, each training technique visibly improves convergence, the "unsupervised meets supervised" concept

articulated. - “If beginners look confused”: “Twenty lines for the E-step earlier today became comfortable. A few hundred lines for a 3-layer network is the natural next step. The scaffolding walks you through forward, backward, and update.” - “If experts look bored”: “The exercise is framework-free on purpose. After you have done backprop by hand once, you will never be confused about what PyTorch is doing under the hood.” **Transition**: “Let us step back and see the full arc.”

Slide 85: Module 4 — The Complete Arc

Time: ~3 min **Talking points**: - Return to the Feature Engineering Spectrum diagram one final time. Manual → USML → DL. - Walk the table row by row. Each lesson produces features of a specific kind: cluster labels (4.1), soft probabilities (4.2), components (4.3), anomaly scores (4.4), rule features (4.5), topic proportions (4.6), embeddings (4.7 — the pivot), hidden activations (4.8). - Point to the Kailash column: AutoMLEngine for clustering, GMM, topics; EnsembleEngine for anomaly detection; ModelVisualizer for dimensionality reduction; OnnxBridge for neural networks. Every lesson had a matching engine. - Deliver the arc in one sentence: “We started by finding groups of similar points. We ended by building a network whose hidden layers discover features for any task. Same spectrum, increasing automation.” - “If beginners look confused”: “If you can point to each row in this table and describe what it produces, you have understood Module 4. That is the check.” - “If experts look bored”: “Modern self-supervised learning, contrastive methods, and foundation model pretraining are all points further right on this spectrum. The thread from K-means to GPT is literally the line drawn on this slide.” **Transition**: “How you will be assessed.”

Slide 86: Module 4 Assessment

Time: ~2 min **Talking points**: - Quiz topics: clustering algorithm selection, EM convergence properties, PCA vs t-SNE vs UMAP, anomaly detection methods, association rule metrics, topic coherence evaluation, matrix factorisation and the pivot, neural network training toolkit. - Project: end-to-end pipeline. (1) Cluster the data. (2) Reduce dimensions. (3) Detect anomalies. (4) Build a neural network on the discovered features. (5) Compare whether USML preprocessing improves DL. - Assessment pedagogy: “The project forces you to use USML output as input to DL. That is the spectrum in practice. If your neural network trains better on clustered + PCA’d features, you have proven that USML is automated feature engineering.” - “If beginners look confused”: “The project is a pipeline with five steps. Do them in order. Each step uses the output of the previous one.” **Transition**: “Looking ahead to Module 5.”

Slide 87: Looking Ahead — Module 5

Time: ~2 min **Talking points**: - M5: LLMs, AI Agents, and RAG Systems. The neural networks from 4.8 scaled to billions of parameters. - Transformers: attention mechanism and positional encoding. Attention is — conceptually — a soft clustering of tokens. Another point on the spectrum. - Large language models: GPT architecture, fine-tuning. These are 4.8’s building blocks stacked very deep on very much data. - AI agents: autonomous systems that use LLMs as reasoning engines. - RAG: retrieval-augmented generation — combine LLM reasoning with external knowledge. - Thread: “4.8’s hidden layers → M5’s transformer layers → attention is soft clustering of tokens → the spectrum never ends.” - “If beginners look confused”: “M5 is the modern AI module. Everything ChatGPT and Claude are built on. You already know the foundations — today’s neural network from scratch is the same math, just much bigger.” - “If experts look bored”: “The interesting bridge for M5 is that attention is a learned kernel function on token embeddings, and multi-head attention is a soft factorisation of the token co-occurrence structure. We will derive it next module.” **Transition**: “One final slide.”

Slide 88: Module 4 Complete

Time: ~1 min **Talking points**: - Read the provocation: “The features that matter most are the ones no human thought to create.” - Thank the class. Remind them of the end-of-module project deadline. - Summary line: “You crossed the bridge today. You arrived this morning with hand-crafted features. You leave this evening having built a neural network that discovers features on its own. That is the bridge from classical ML to modern AI, and you walked across it in three hours.” - Optional closing: “If you want to see the spectrum continue, Module 5 scales what you just built to the models that power every frontier AI system today. See you next week.” **Transition**: [End of module]